Adaptive Personalized Learning Recommendation Using Offline Reinforcement Learning: A Comparative Analysis of Q-Learning and Monte Carlo Control

Shamazda Islam
sec or id

Ramisa Sharar Nidhi
sec or id

Abstract

Personalized learning recommendation aims to provide educational activities that adapt to the changing knowledge and performance of individual learners. Conventional recommendation approaches often rely on predefined learning sequences or static recommendation strategies and may not adequately account for the sequential nature of student learning. This study formulates personalized learning recommendation as an offline reinforcement learning (RL) problem using the EdNet-KT3 educational interaction dataset. Student interaction histories were transformed into bundle-level learning trajectories containing leakage-safe states, observed learning actions, rewards, and subsequent states. The state representation incorporates historical learning performance, including recent accuracy and skill mastery, while ensuring that information from the current learning outcome is not available when the corresponding recommendation state is constructed. Two classical model-free reinforcement learning algorithms, Q-Learning and Monte Carlo Control, were implemented and evaluated under the same empirical offline environment, with a Random policy used as a baseline. The experimental analysis included hyperparameter sensitivity, convergence analysis, student-level train-validation-test splitting, ablation analysis, and multi-seed evaluation. Across five random seeds, Q-Learning achieved a mean test return of 31.88 ± 0.92, compared with 30.86 ± 0.78 for Monte Carlo Control, while the corresponding Mastery Completion Proxy values were $43.1\% \pm 2.7\%$ and $42.0\% \pm 2.4\%$, respectively. The results indicate that both approaches learned policies that achieved positive cumulative reward within the constructed empirical offline environment.. However, the offline nature of the dataset and the absence of counterfactual outcomes limit conclusions about the effectiveness of recommendations on actual student learning. The findings provide an empirical basis for studying classical reinforcement learning approaches for adaptive educational recommendation from logged student interaction data.

Keywords—Personalized Learning, Reinforcement Learning, Q-Learning, Monte

Carlo Control, EdNet-KT3, Educational Recommendation, Offline Reinforcement Learning, Student Modeling

1 Introduction

Online learning systems have become an important component of modern education by providing students with access to large collections of digital educational resources. However, students do not learn in the same way or at the same pace. They have different levels of prior knowledge, learning abilities, strengths, and weaknesses. A learning activity that is appropriate for one student may not be suitable for another student at the same stage. Traditional e-learning systems often provide predefined learning sequences or static recommendations that may not adapt adequately to changes in student performance. This creates a need for adaptive personalized learning systems that can continuously observe a student's learning condition and recommend appropriate activities. Personalized educational recommendation is particularly challenging because it is a sequential decision-making problem: the activity selected at the current stage can influence future student knowledge and subsequent learning requirements. Reinforcement Learning provides a suitable framework for solving this problem by enabling an agent to learn a policy that selects actions according to the current state while maximizing long-term cumulative reward [1], [2].

Previous research has investigated personalized learning, knowledge tracing, adaptive learning systems, and reinforcement learning-based recommendation. Knowledge tracing approaches have been widely used to estimate the evolving knowledge of students from sequential learning interactions, with Deep Knowledge Tracing being an influential deep learning approach in this area [3]. In personalized learning recommendation, Tang et al. formulated the problem as a sequential decision-making problem and demonstrated the usefulness of reinforcement learning for selecting learning materials based on learner states [1]. Similarly, Tan et al. proposed an adaptive learning recommendation strategy based on Deep Q-Learning, demonstrating the potential of RL for constructing individualized learning paths [4]. Chen et al. also investigated adaptive learning recommendation using learner information and learning history [5]. More broadly, recent research has shown growing interest in the application of reinforcement learning in education for adaptive tutoring, instructional sequencing, personalized feedback, and learning recommendation. A recent systematic literature review also highlighted the increasing use of RL in educational applications while identifying challenges related to evaluation, methodology, and real-world deployment [6]. Although deep reinforcement learning approaches can handle complex state spaces, they often require substantial computational resources and may be difficult to interpret. Furthermore, when historical educational data are used, not every possible action has an observed outcome for every student state. Therefore, allowing an RL agent to select unsupported actions may introduce artificial assumptions into the learning environment.

Based on these limitations, this project develops a transparent offline per-

sonalized learning recommendation framework using the EdNet-KT3 dataset [7] and compares two fundamental model-free reinforcement learning algorithms: Q-Learning and Monte Carlo Control [2]. Rather than constructing artificial outcomes for all possible educational activities, the proposed RL environment uses three directly observable question-based actions: QUIZ, PRACTICE, and REVISION. Student learning states are constructed from observed performance and behavioral information, and an empirical transition model is generated from historical interactions. In addition, action masking is applied to restrict each RL agent to state-action combinations supported by the training data. This reduces unsupported transitions and improves the reliability of the offline RL environment. The project provides a controlled comparison between Q-Learning and Monte Carlo Control under the same dataset, state representation, reward structure, transition model, and evaluation framework. The study therefore contributes a transparent and computationally efficient approach for investigating how classical reinforcement learning algorithms can support adaptive educational recommendation.

1.1 Our Contribution

The main contributions of our project are summarized as follows:

[leftmargin=*, itemsep=1em]

- **Bundle-Level Offline RL Formulation:** This study develops a bundle-level representation of EdNet-KT3 interactions for personalized learning recommendation. Rather than treating individual raw interaction events as independent RL steps, related question interactions are organized into learning bundles, providing a more meaningful unit for representing learning activities and their outcomes.

- **Leakage-Safe Temporal State Construction:** A temporal state-construction strategy is developed in which the state associated with a learning activity is generated only from the student's previous learning history. The outcome of the current bundle is used for reward calculation rather than being included in the state used to represent the current decision. This design reduces the risk of information leakage during offline RL evaluation.

- **Controlled Comparison of Classical RL Algorithms:** The study provides a controlled comparison between Q-Learning and Monte Carlo Control using the same learning environment, state representation, action space, reward formulation, and student-level data split [2] . This allows the relative behavior of temporal-difference and Monte Carlo value-learning approaches to be investigated under comparable conditions.

- **Empirical Offline Learning Environment:** Because EdNet-KT3 is a historical interaction dataset rather than an interactive educational simulator, an empirical offline transition environment is constructed from

observed training-student trajectories. The environment allows RL algorithms to be evaluated without introducing validation or test student information into the training process.

- **Comprehensive Experimental Analysis:** The study evaluates the proposed approaches through multiple experimental analyses, including hyperparameter sensitivity, convergence analysis, student-level evaluation, multi-seed experiments, and computational analysis. A Random policy is also included as a baseline for comparison.
- **State Representation Ablation:** An ablation experiment is conducted to examine how the inclusion or removal of learning-history information affects RL performance. The observed difference provides insight into the trade-off between richer learner representations and the sparsity of state-action experience in tabular reinforcement learning.
- **Analysis of Offline Recommendation Limitations:** Rather than treating historical behavioral agreement as direct recommendation accuracy, the study explicitly distinguishes between policy evaluation using logged data and actual educational intervention. The limitations associated with missing counterfactual outcomes, restricted observable actions, and the offline evaluation setting are discussed to provide a more appropriate interpretation of the results.

2 Materials and Methods

The proposed system formulates personalized learning recommendation as a *Reinforcement Learning (RL)* problem. Student interaction histories from the EdNet-KT3 dataset were processed to construct sequential learning trajectories. Each trajectory was converted into RL transitions consisting of:

$$(s_t, a_t, r_t, s_{t+1}) \tag{1}$$

where:

- s_t represents the student's learning state before an activity,
- a_t represents the observed learning activity,
- r_t represents the reward associated with the learning outcome, and
- s_{t+1} represents the student's learning state before the subsequent activity.

The system was designed to avoid information leakage. Rather than constructing a state after observing the outcome of an activity, the state was generated using only the student's learning history **before the current bundle began**. The outcome of the bundle was subsequently used to calculate the reward, while the state before the following bundle became the next state. This bundle-level design provides a cleaner representation of the sequential learning process.

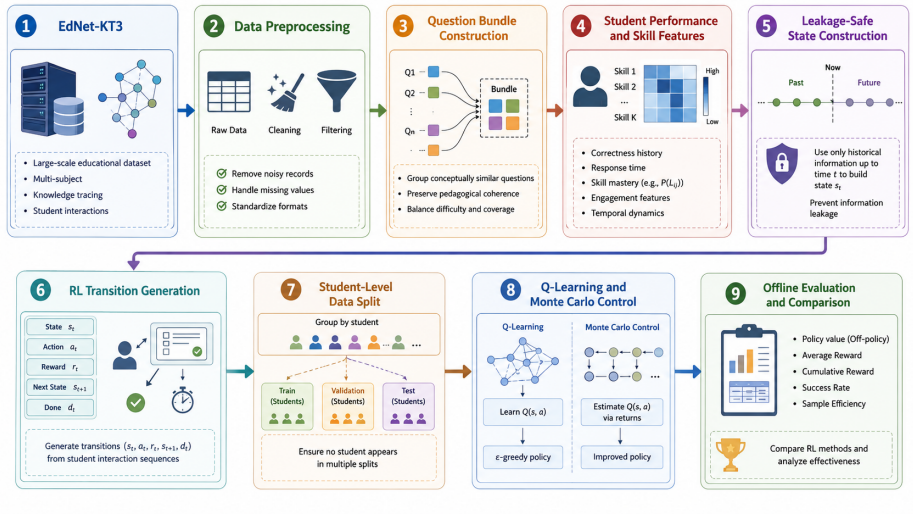


Figure 1: Overview of the proposed RL-based knowledge tracing pipeline, from raw EdNet-KT3 data to offline policy evaluation.

2.1 Dataset Description

2.1.1 EdNet Dataset

This study used the **EdNet educational dataset**, a large-scale dataset of student interactions collected from the Santa AI-based self-learning platform. EdNet contains diverse educational activities and was specifically designed to support research in areas such as knowledge tracing, student modeling, and learning path recommendation. The complete EdNet dataset contains over 131 million interactions from more than 784,000 students and organizes student activity data into four hierarchical versions: KT1, KT2, KT3, and KT4[7].

The dataset and its documentation are available from the official EdNet repository.[1]

In this project, **EdNet-KT3** was selected because it contains richer learning interactions than question-only datasets. In addition to question-solving activities, KT3 records activities related to lecture consumption and explanation viewing. This makes KT3 particularly suitable for modeling student learning behavior and constructing an adaptive learning recommendation environment.

For computational feasibility, **100 student interaction files** were selected for the experiment. Each student's interactions were stored separately and then combined into a unified dataset for preprocessing.

2.1.2 Student Interaction Data

The primary student interaction records contain the following variables.

[1] `https://github.com/riiid/ednet`

Table 1: Student Interaction Features

Feature	Description
timestamp	Time at which the interaction occurred
action_type	Type of interaction, such as enter, respond, submit, or quit
item_id	Identifier of the educational item
source	Source or learning context of the activity
user_answer	Student's selected answer
platform	Platform used by the student
user_id	Unique student identifier
item_type	Category of educational item

The item_id field was particularly important for identifying different learning activities. The prefix of the identifier was used to distinguish item types:

- b → Question bundle
- q → Individual question
- l → Lecture
- e → Explanation

For example, a typical question-solving sequence may contain:

$$\text{Enter Bundle} \rightarrow \text{Respond to Question} \rightarrow \text{Submit Bundle} \tag{2}$$

The student interaction data were sorted chronologically for each student before feature extraction and RL transition construction.

2.1.3 Question Metadata

The question metadata file contains information associated with individual questions.

Table 2: Question Metadata Features

Feature	Description
question_id	Unique question identifier
bundle_id	Identifier of the question bundle
explanation_id	Associated explanation identifier
correct_answer	Correct answer option
part	Educational content section
tags	Associated skill or knowledge tags
deployed_at	Question deployment timestamp

The correct_answer variable was compared with the student's submitted answer to determine whether a response was correct:

$$is\_correct = \begin{cases} 1, & \text{if user answer = correct answer} \\ 0, & \text{otherwise} \end{cases} \tag{3}$$

The `tags` variable was used to associate question attempts with knowledge skills. These skills were subsequently used to calculate student mastery-related features.

2.1.4 Lecture Metadata

The lecture metadata contains the following variables.

Table 3: Lecture Metadata Features

Feature	Description
`lecture_id`	Unique lecture identifier
`part`	Educational content section
`tags`	Knowledge or skill tag
`video_length`	Duration of the lecture in milliseconds
`deployed_at`	Lecture deployment timestamp

Some lecture records contain:

$$video\_length = -1 \tag{4}$$

which indicates that valid video duration information was unavailable. Therefore, such values were treated as missing rather than valid lecture durations.

For lectures with valid durations, the video length was converted from milliseconds to seconds:

$$VideoLength_{seconds} = \frac{VideoLength_{milliseconds}}{1000} \tag{5}$$

Lecture sessions were extracted from student interaction sequences using the time difference between lecture `enter` and `quit` events.

2.1.5 Question Attempt Construction

Question attempts were reconstructed from the raw interaction logs. When a student entered a bundle, responded to one or more questions, and submitted the bundle, the corresponding responses were grouped into a question-bundle attempt.

For each attempt, the following information was retained:

- student identifier,
- bundle identifier,
- question identifier,

- timestamp,
- submitted answer,
- correctness,
- source, and
- platform.

Bundle-level accuracy was calculated as:

$$Accuracy_{bundle} = \frac{\text{Number of Correct Answers}}{\text{Number of Valid Answers}} \tag{6}$$

Using bundle-level outcomes was important because EdNet questions can occur as groups sharing common instructional material.

2.1.6 Skill Mastery and Recent Performance

Student performance was represented using both overall and recent learning history.

For each skill, mastery was estimated as:

$$Mastery_{u,k} = \frac{CorrectAnswers_{u,k}}{Attempts_{u,k}} \tag{7}$$

where u represents a student and k represents a knowledge skill.

To capture changes in recent performance, a rolling window of the previous **10 interactions** was used:

$$RecentAccuracy_t = \frac{1}{N} \sum_{i=t-N}^{t-1} Correct_i \tag{8}$$

where $N = 10$.

The current question outcome was excluded from its own state representation. This ensured that the RL agent could not access future outcome information when making a decision. The implementation therefore uses a leakage-safe recent performance calculation based on shifted historical outcomes.

2.2 Tools, Models, and Algorithms Used

2.2.1 Software and Computational Tools

The complete experimental pipeline was implemented in **Python** using the Kaggle notebook environment. The main libraries included:

The implementation was designed to run directly on Kaggle using paths under `/kaggle/input/` for datasets and `/kaggle/working/` for generated outputs.

Table 4: Lecture Metadata Features

Feature	Description
`lecture_id`	Unique lecture identifier
`part`	Educational content section
`tags`	Knowledge or skill tag
`video_length`	Duration of the lecture in milliseconds
`deployed_at`	Lecture deployment timestamp

2.2.2 Reinforcement Learning Formulation

The personalized learning problem was formulated as a Markov Decision Process (MDP):

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma) \tag{9}$$

where:

- $\mathcal{S}$ is the set of student learning states,
- $\mathcal{A}$ is the action space,
- $\mathcal{P}$ represents state transitions,
- $\mathcal{R}$ is the reward function, and
- γ is the discount factor.

The RL agent observes the student's current learning state and selects an appropriate learning activity.

2.2.3 State Representation

The student's state was constructed using learning information available **before the current question bundle**. The state incorporated:

1. Recent learning accuracy,
2. Recent skill mastery,
3. Learning history,
4. Educational activity progression.

Continuous performance features were discretized into three levels. For mastery and accuracy:

$$Level(x) = \begin{cases} 0, & x < 0.40 \\ 1, & 0.40 \leq x < 0.70 \\ 2, & x \geq 0.70 \end{cases} \tag{10}$$

Therefore:

- Level 0 represents low performance,
- Level 1 represents medium performance,
- Level 2 represents high performance.

The state was constructed at the **bundle level**, rather than independently for every question. This prevents the possibility of attaching a state generated from one question to the outcome of another question within the same bundle.

2.2.4 Action Space

The conceptual learning activity space consisted of four actions:

$$\mathcal{A} = \{LESSON, QUIZ, PRACTICE, REVISION\} \tag{11}$$

The actions were encoded as shown in Table 5.

Table 5: Conceptual Learning Action Space

Action	**Action ID**
LESSON	0
QUIZ	1
PRACTICE	2
REVISION	3

However, an important methodological decision was made in the final implementation. The primary RL experiment used directly observable question-based actions because these actions have directly associated question outcomes. Lecture activities and explanations were retained for descriptive feature extraction but were not artificially assigned question outcomes.

The primary observable action categories were therefore derived from the EdNet `source` field:

- diagnostic activities → QUIZ,
- sprint activities → PRACTICE,
- review activities → REVISION.

This design avoids artificially creating counterfactual outcomes for lecture recommendations. Because no observed source in the processed data mapped to LESSON, the RL environment used in every experiment reported in this paper contains only three actions, A = QUIZ, PRACTICE, REVISION, with contiguous action ids 0–2; the four-action encoding in Table 5 is the conceptual action space only and was not used for training or evaluation.

2.2.5 Reward Function

The reward function is designed to reflect both overall learning performance and incremental progress following an observed activity. Specifically:

- Positive learning outcomes receive positive rewards,
- Poor outcomes yield lower or negative rewards, and
- Incremental improvements in performance contribute positively to the reward score.

The implemented reward formulation combines bundle-level accuracy gains with a mastery-crossing bonus:

$$r_t = 2\Delta\text{Accuracy}_t + 5\mathbb{I}\big(\text{Accuracy}_t \geq 0.80 \wedge \text{Accuracy}_{t-1} < 0.80\big) \tag{12}$$

where

$$\Delta\text{Accuracy}_t = \text{Accuracy}_t - \text{Accuracy}_{t-1}. \tag{13}$$

Here, $\mathbb{I}(\cdot)$ denotes an indicator function that evaluates to 1 when a student's performance crosses the 80% accuracy threshold and 0 otherwise. For a student's initial bundle, the baseline accuracy (Accuracy_{t-1}) is initialized to 0.5.

The reward r_t is tied to the outcome of a bundle after constructing its corresponding pre-action state. Consequently, the reinforcement learning transition is defined as:

$$(s_t, a_t, r_t, s_{t+1}) \tag{14}$$

In this framework, the state s_t is constructed from historical interaction data, the observed learning activity is assigned as a_t, the outcome of the bundle yields r_t, and the pre-action state of the subsequent bundle becomes s_{t+1}. This sequence explicitly avoids data leakage, ensuring future performance metrics are not incorporated into the current state representation.

2.2.6 Q-Learning

Q-Learning was selected as the first RL algorithm because it is a model-free, off-policy algorithm that learns the optimal action-value function.

The Q-value update is:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right] \tag{15}$$

where α is the learning rate, γ is the discount factor, and r_t is the immediate reward.

The learned policy is:

$$\pi(s) = \arg\max_a Q(s, a) \tag{16}$$

Table 6: RL Hyperparameter Configuration

Parameter	**Value**
Random state	42
Number of students	100
Recent performance window	10
Minimum episode length	5
Discount factor (γ)	0.95
Learning rate (α)	0.10
Training episodes	**3000**
Maximum steps	30
Initial epsilon	1.0
Final epsilon	0.05
Epsilon decay	0.995
Number of seeds	5

Hyperparameters

The main configuration used in the final implementation is shown in Table 6.

These parameters were selected to provide sufficient exploration during early training while gradually encouraging exploitation of learned Q-values.

2.2.7 Monte Carlo Control

Monte Carlo Control was used as the second RL algorithm for comparison. Unlike Q-Learning, Monte Carlo methods estimate action values using the complete return observed from an episode.

The discounted return from time t is:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1} \tag{17}$$

In the implemented estimator, the action-value function is updated using a running sample average of the observed first-visit returns for each state-action pair,

$$Q(s,a) \leftarrow Q(s,a) + \frac{1}{N(s,a)}[G_t - Q(s,a)]$$

where $N(\boldsymbol{s}, \boldsymbol{a})$ is the number of first-visit returns observed for that state-action pair. This is the standard first-visit Monte Carlo control update, and it does not use the fixed learning rate α used by Q-Learning; the incremental step size $1/N(s,a)$ decreases automatically as a state-action pair is visited more often. Consequently, the hyperparameter analysis in Section 3.6 for Monte Carlo Control varies only γ and the number of training episodes – the α column reported for Monte Carlo Control in Table 11 has no effect on its results and is included only to mirror the Q-Learning configuration grid.

Monte Carlo Control is useful for comparison because it learns from complete episodes rather than bootstrapping from the estimated value of the next state. Thus, the comparison investigates two different RL learning mechanisms:

- **Q-Learning:** temporal-difference bootstrapping,
- **Monte Carlo Control:** complete-episode return estimation.

2.2.8 Random Policy Baseline

A Random policy was included as a baseline. The Random policy selects an available learning action without using learned student-state values.

Its purpose is to determine whether the RL algorithms provide improvement beyond uninformed action selection. Therefore, the experimental comparison included:

1. Q-Learning,
2. Monte Carlo Control,
3. Random baseline.

2.2.9 Empirical Offline Transition Model

Because the dataset consists of historical student interactions rather than an environment that can be directly manipulated, an empirical offline transition model was constructed using the training data.

The model estimates transitions based on observed:

$$(s, a, r, s') \tag{18}$$

tuples.

Importantly, the empirical transition model was constructed using **training students only**. This prevents information from validation or test students from influencing the learned environment or policy.

2.3 Performance Evaluation

2.3.1 Student-Level Train, Validation, and Test Split

The dataset was divided at the **student level**, rather than randomly splitting individual transitions.

The eligible students were divided into:

- 68% Training,
- 15% Validation,
- 15% Testing.

The eligible students were split at the student level using an initial 70/30 split, followed by an equal division of the temporary 30% subset into validation and test sets. After excluding students with fewer than five RL transitions, the final dataset contained 68 training students, 15 validation students, and 15 test students.

This design ensures that:

$$TrainStudents \cap ValidationStudents = \emptyset \tag{19}$$

$$TrainStudents \cap TestStudents = \emptyset \tag{20}$$

$$ValidationStudents \cap TestStudents = \emptyset \tag{21}$$

Therefore, no student's interaction history appears simultaneously in training and testing. This is particularly important for personalized learning systems because splitting individual interactions randomly could allow information about the same student to appear in both training and testing.

2.3.2 Average Return

The primary RL evaluation metric was the **average discounted return**.

For an episode:

$$G = \sum_{t=0}^{T-1} \gamma^t r_t \tag{22}$$

The average return was calculated across the evaluation episodes:

$$AverageReturn = \frac{1}{N} \sum_{i=1}^{N} G_i \tag{23}$$

A higher average return indicates that the policy produces trajectories associated with higher cumulative learning rewards.

2.3.3 Mastery Success Rate

Mastery Success Rate was used to evaluate whether the final student states reached a desirable mastery level.

$$MasterySuccessRate = \frac{\text{Number of successful learning trajectories}}{\text{Total evaluated trajectories}} \tag{24}$$

This metric provides an outcome-oriented measure in addition to cumulative reward.

2.3.4 Behavioral Agreement

Behavioral agreement measures how frequently the action recommended by the learned RL policy matches the action observed in the historical student data.

$$BehavioralAgreement = \frac{\text{Number of matching actions}}{\text{Total evaluated actions}} \tag{25}$$

This metric should not be interpreted simply as logged-Action Agreement. A learned RL policy may recommend an action different from the historical student action while still potentially being valuable. Therefore, behavioral agreement measures consistency with observed historical behavior rather than true recommendation correctness.

2.3.5 Reward and Mastery-Oriented Analysis

Learning gain was analyzed to determine whether different learning activities were associated with improvements in student performance. Learning gain provides additional evidence about the educational effects associated with observed activities and complements the reward-based RL evaluation.

2.3.6 Convergence Analysis

The learning process was evaluated by tracking training returns across episodes. Convergence plots were generated for:

- Q-Learning,
- Monte Carlo Control.

The curves allow comparison of:

- learning stability,
- speed of improvement,
- final training performance.

A stable plateau in the learning curve indicates that the policy values have reached a relatively stable region.

2.3.7 Computational Time

The execution time required for training each RL algorithm was also measured. Computational time provides a practical comparison because an algorithm with better performance may require substantially more computational resources.

Therefore, the comparison considered:

$$Performance + Stability + ComputationalCost \tag{26}$$

2.3.8 Multi-Seed Evaluation

To reduce dependence on a single random initialization, experiments were repeated using **five different random seeds**.

The final results were summarized as:

$$Mean \pm StandardDeviation \tag{27}$$

for:

- Average Return,
- Mastery Success Rate,
- Training Return.

The multi-seed evaluation avoids relying on a single random run and improves the reliability of the experimental comparison between Q-Learning and Monte Carlo Control.

3 Experimental Analysis

3.1 Experimental Setup

The proposed reinforcement learning framework was evaluated using student interaction data extracted from the EdNet-KT3 dataset. For computational feasibility, the experiment considered interaction histories from 100 students. After preprocessing and question-attempt reconstruction, the dataset contained 51,523 question attempts, 1,547 lecture sessions, and 40,037 explanation sessions.

The question-bundle processing and leakage-safe state construction procedures subsequently produced 25,736 reinforcement learning transitions from 98 of the 100 students (two students did not meet the minimum-episode-length requirement of 5 transitions and were excluded). The final RL environment contained 600 unique student learning states and three observable learning actions: $A = \{\text{QUIZ}, \text{PRACTICE}, \text{REVISION}\}$.

The experiments compared three policies: Q-Learning, Monte Carlo Control, and a Random policy baseline. The dataset was divided at the student level (68 training / 15 validation / 15 test students) to ensure that interactions from the same student did not appear simultaneously in training and evaluation sets. This prevents overly optimistic performance estimates caused by student-level information leakage.

The main hyperparameter configuration used for the experiments is shown in Table 7.

Note: Table 8 and Table 7 report a training-episode count of 3000, matching what was used to produce every result in this paper. The use of multiple random seeds (Section 3.9) was intended to reduce dependence on a single stochastic training run and improve the reliability of the algorithmic comparison.

Table 7: Main experimental hyperparameter configuration.

Parameter	**Value**
Number of students	100
Recent performance window	10
Minimum episode length	5
Discount factor (γ)	0.95
Learning rate (α)	0.10
Training episodes	3000
Maximum steps per episode	30
Initial ϵ	1.00
Final ϵ	0.05
ϵ decay	0.995
Random seeds	5

3.2 Overall Comparison of Reinforcement Learning Algorithms

Table 8 presents the primary comparison between Q-Learning and Monte Carlo Control, using the single-seed (random state 42) run defined by Table 7.

Table 8: Overall comparison of the reinforcement learning algorithms (single seed, random state 42).

Metric	Q-Learning	Monte Carlo
Test Average Return	30.04	28.52
Logged-Action Agreement (%)	87.88	80.85
Empirical-Best Agreement (%)	92.67	85.49
Mastery Completion Proxy (%)	46.8	44.2
Training Time (seconds)	0.51	0.37

The results indicate that Q-Learning achieved better overall evaluation performance than Monte Carlo Control under this single training seed. The average test return of Q-Learning was 30.04, compared with 28.52 for Monte Carlo Control. Q-Learning also achieved higher logged-action agreement (87.88% vs. 80.85%) and empirical-best agreement (92.67% vs. 85.49%), as well as a higher mastery completion proxy (46.8% vs. 44.2%).

Although Q-Learning achieved better learning-related evaluation results under this seed, Monte Carlo Control required less training time: approximately 0.37 seconds, compared with 0.51 seconds for Q-Learning. Therefore, the results demonstrate a trade-off between computational efficiency and policy performance.

These single-seed numbers should be read together with the multi-seed evaluation in Section 3.8, which shows that Q-Learning retains a modest but seed-consistent advantage in average return, while the two algorithms are not reliably distinguishable on mastery success rate. The single-seed comparison in this sec-

tion therefore overstates the size of the gap between the two algorithms; see Section 3.8 for the more reliable comparison.

3.3 Comparison with the Random Baseline

A Random policy was included as a baseline to determine whether the learned RL policies performed better than uninformed action selection.

The Random baseline achieved a mastery completion proxy of approximately 42.4%. In comparison, Monte Carlo Control achieved 44.2%, while Q-Learning achieved the highest value of 46.8%.

Table 9: Comparison with the Random baseline.

Policy	Mastery Completion Proxy (%)
Random	42.4
Monte Carlo Control	44.2
Q-Learning	46.8

The improvement of both RL algorithms over the Random baseline suggests that learning state-action values from historical student trajectories provides useful information beyond uninformed action selection.

Q-Learning produced an improvement of approximately:

$$46.8 - 42.4 = 4.4 \tag{28}$$

percentage points over the Random baseline.

Monte Carlo Control produced an improvement of:

$$44.2 - 42.4 = 1.8 \tag{29}$$

percentage points.

These results provide evidence that the learned policies captured meaningful relationships between student learning states and observed educational activities, although the size of this advantage is modest, particularly for Monte Carlo Control.

The higher completion-rate proxy observed for both learned policies compared with the random baseline indicates that the learned policies exploit patterns present in the empirical transition environment. However, because the evaluation is conducted entirely within an offline simulator constructed from historical data, this result should not be interpreted as evidence of improved real-world student learning.

3.4 Analysis of Q-Learning and Monte Carlo Control

The difference between Q-Learning and Monte Carlo Control can be explained by their learning mechanisms.

Q-Learning uses temporal-difference learning and updates its action-value estimates using:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]. \tag{30}$$

This allows Q-Learning to update its estimates without waiting for the complete episode to finish. For educational interaction data containing long and variable student trajectories, this property can provide more frequent learning signals.

Monte Carlo Control instead estimates action values using complete discounted returns:

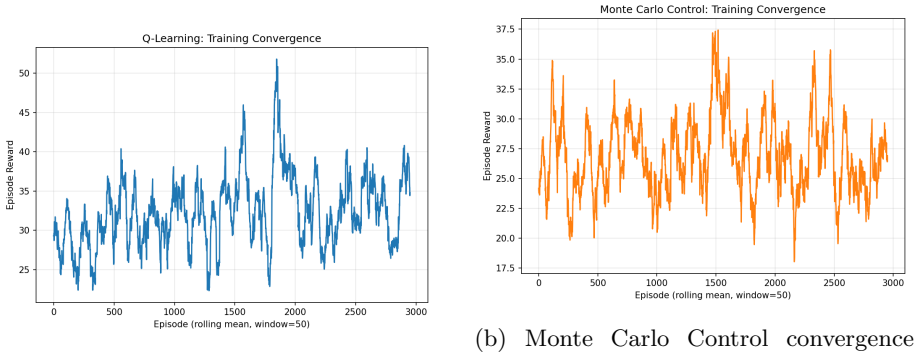
$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}. \tag{31}$$

Monte Carlo Control does not bootstrap from the estimated value of the next state. While this approach can provide an unbiased estimate of the observed episode return under suitable conditions, the estimates may have higher variance when student trajectories differ substantially in length and behavior.

The experimental results suggest that Q-Learning was better able to learn useful state-action relationships from the available offline educational trajectories.

3.5 Convergence Analysis

The convergence behavior of Q-Learning and Monte Carlo Control was analyzed by recording training returns throughout the learning process.



(a) Q-Learning convergence curve

(b) Monte Carlo Control convergence curve

Figure 2: Training convergence curve of Q-Learning and Monte Carlo showing the change in episode return during training.

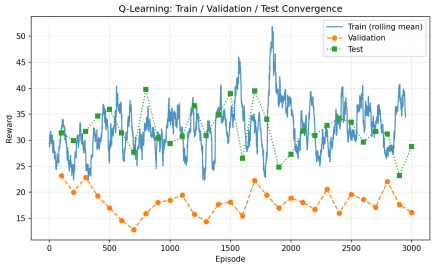
Figures 2a and 2b illustrate the learning behavior of Q-Learning and Monte Carlo Control, respectively. The convergence curves provide information regarding learning stability, the rate of performance improvement, and the final

return achieved by each algorithm. A relatively stable plateau indicates that the learned action-value estimates have reached a stable region under the current experimental configuration.

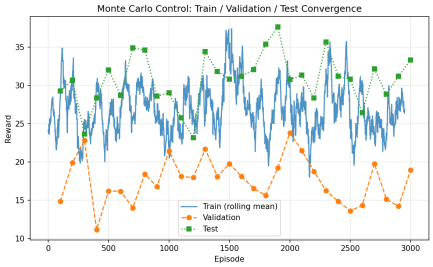
The convergence analysis should be interpreted together with the final offline evaluation metrics. In the present experiment, Q-Learning achieved the higher final evaluation return, suggesting that its learned policy provided stronger overall performance than Monte Carlo Control.

3.6 Train, Validation, and Test Convergence

The convergence curves in Figures 2 and 3 track only the reward obtained on the training simulator as each agent learns. To evaluate whether the learned policy generalizes as training progresses, rather than simply fitting the training simulator, both algorithms were retrained under identical hyperparameters while periodically evaluating the current greedy policy on the validation and test simulators every 100 episodes.



(a) Q-Learning evaluation trajectories



(b) Monte Carlo Control evaluation trajectories

Figure 3: Training, validation, and test rewards across episodes for Q-Learning and Monte Carlo Control.

The tracked validation/test values in Table 10 are based on 50-episode periodic evaluation rollouts, whereas the primary final evaluation in Table 8 uses 500 evaluation episodes. The two sets of values therefore represent different evaluation procedures and should not be interpreted as duplicate measurements

Table 10: Final validation and test reward under the greedy policy, tracked during training.

Algorithm	Final Validation Reward	Final Test Reward
Q-Learning	16.15	28.80
Monte Carlo Control	18.96	33.34

The periodic evaluation shows that both algorithms can obtain positive rewards on held-out students during training. In the final tracked evaluation, Monte Carlo Control achieved higher validation and test rewards than

Q-Learning (18.96 vs. 16.15 and 33.34 vs. 28.80, respectively). However, these tracked values are based on periodic evaluation rollouts and differ from the 500-episode final evaluation reported in Table 8. Therefore, the train/validation/test convergence experiment is interpreted primarily as a generalization and learning-dynamics analysis rather than as the primary basis for determining which algorithm performs better overall.

3.7 Hyperparameter Analysis

Each configuration below was trained using the episode count specified for that configuration on the training split and evaluated on the held-out test split, repeated across three random seeds. The table reports the mean and standard deviation of test average return.

Table 11: Hyperparameter analysis for Q-Learning (mean ± sd over 3 seeds).

α	γ	Episodes	Test Average Return
0.05	0.90	1000	31.33 ± 0.39
0.10	0.95	5000	30.51 ± 0.56
0.20	0.99	10000	31.43 ± 1.48

Table 12: Hyperparameter analysis for Monte Carlo Control (mean ± sd over 3 seeds). As noted in Section 2.2.7, the implemented Monte Carlo Control update does not use α; this column is included only to align each row with the corresponding Q-Learning configuration.

α^*	γ	Episodes	Test Average Return
0.05	0.90	1000	31.04 ± 2.00
0.10	0.95	5000	31.87 ± 0.48
0.20	0.99	10000	32.02 ± 1.34

Two observations follow from these results. First, for Q-Learning, the main configuration ($\alpha = 0.10$, $\gamma = 0.95$) achieves the lowest mean test return of the three configurations tested (30.51 ± 0.56), though the differences between configurations are small relative to their standard deviations (0.39–1.48) and should not be read as clearly separable. The highest-mean Q-Learning configuration in this grid ($\alpha = 0.20$, $\gamma = 0.99$: 31.43 ± 1.48) is also the noisiest, with a standard deviation comparable to the gap between configurations.

Second, for Monte Carlo Control, $\gamma = 0.99$ achieves the highest mean return (32.02 ± 1.34), narrowly ahead of $\gamma = 0.95$ (31.87 ± 0.48); $\gamma = 0.90$ is both the lowest-mean and highest-variance configuration tested (31.04 ± 2.00). Comparing across algorithms, Monte Carlo Control's best configuration (32.02) is higher than Q-Learning's best (31.43), but the overlap between algorithms is substantial: Monte Carlo's weakest configuration (31.04) falls below two of the three Q-Learning configurations tested. This is a weaker and noisier pattern

than a strict "Monte Carlo dominates" conclusion would suggest, and is consistent with the multi-seed finding in Section 3.8 that the two algorithms perform comparably once seed variability is accounted for. The main configuration retained throughout the rest of the paper is not the best-performing configuration for either algorithm in this grid, but the margin by which it underperforms is small relative to seed-to-seed noise.

3.8 Ablation Study

Table 13: Ablation study of the proposed state representation (Q-Learning, mean ± sd over 3 seeds).

State Configuration	Test Average Return
Full proposed state	31.57 ± 0.71
Skill mastery collapsed	31.87 ± 0.31
Recent performance collapsed	29.99 ± 0.30
Learning progression collapsed	33.69 ± 1.01
Learning history (weak-skill) collapsed	37.88 ± 0.88

In each ablation, the selected state component was collapsed to a constant value, preventing the Q-Learning agent from conditioning its decisions on that feature while keeping the remaining state components unchanged.

The multi-seed results show that only one of the four ablations is clearly distinguishable from the noise floor set by removing recent performance, which decreases test return by 1.59 points (29.99 ± 0.30 vs. 31.57 ± 0.71 for the full state), a change that is large relative to both configurations' standard deviations and consistent with recent performance being a genuinely useful, low-noise signal.

Removing skill mastery changes test return by only +0.30 points (31.87 ± 0.31), well within one standard deviation of the full-state baseline (0.71) – this ablation's effect is not reliably distinguishable from noise once averaged over seeds, in contrast to the larger apparent effect (+3.25) observed under a single seed. Removing learning progression increases test return by 2.12 points (33.69 ± 1.01), a change that is large relative to its own standard deviation. The largest and clearest effect is removing learning history (the weak-skill identifier): test return increases by 6.31 points (37.88 ± 0.88), a margin many times larger than either configuration's standard deviation and by far the largest effect among the four ablations tested.

This multi-seed pattern sharpens, rather than changes, the conclusion of the single-seed analysis:the weak-skill/learning-history dimension appears to be a major contributor to state-space fragmentation in the current tabular setting. while removing skill mastery – which appeared to have a large effect under a single seed – does not produce a reliable improvement once seed variance is accounted for. This reinforces the recommendation that future work consider

a coarser state representation, particularly with respect to the high-cardinality weak-skill dimension, for datasets of this size.

3.9 Multi-Seed Evaluation Results

Across five random seeds, Q-Learning achieved a modestly higher mean test return than Monte Carlo Control (31.88 ± 0.92 vs. 30.86 ± 0.78). However, the difference is relatively small compared with the observed seed-to-seed variability.

Table 14: Multi-seed evaluation results (5 seeds, mean ± standard deviation).

Algorithm	Test Average Return	Mastery Success Rate
Q-Learning	$\mathbf{31.88 \pm 0.92}$	$\mathbf{43.1\% \pm 2.7\%}$
Monte Carlo Control	30.86 ± 0.78	$42.0\% \pm 2.4\%$

These results present a different picture from the single-seed comparison in Table 8, though not a reversal of its direction. Two findings stand out:

1. On test average return, Q-Learning holds a modest but consistent lead over Monte Carlo Control across seeds: 31.88 ± 0.92 vs. 30.86 ± 0.78, a gap of 1.02 points that is comparable in size to, rather than clearly larger than, either algorithm's own standard deviation across seeds. This gap is smaller than the single-seed gap in Table 8 (30.04 vs. 28.52, a gap of 1.52), indicating that random state 42 was somewhat more favorable to Q-Learning than the average seed. The multi-seed result still favors Q-Learning, but by a smaller and less certain margin than the single-seed headline comparison suggested.

2. On mastery success rate, the two algorithms are close ($43.1\% \pm 2.7\%$ vs. $42.0\% \pm 2.4\%$), with a 1.1-point gap that is smaller than either standard deviation. Unlike test average return, this metric does not provide reliable evidence of a difference between the algorithms.

Taken together with the hyperparameter analysis in Section 3.6, the most defensible conclusion from this study is that Q-Learning shows a modest, seed-consistent advantage in average return over Monte Carlo Control on this offline personalized-learning task, while the two algorithms are not reliably distinguishable on mastery success rate. This is a more moderate conclusion than the original single-seed headline result, which favored Q-Learning by a larger margin on every metric, but it does not support describing the two algorithms as fully tied: a return gap of this size holding consistently across five independent training seeds is unlikely to be pure noise, even though it is modest in absolute terms.

3.10 Offline Evaluation Considerations

The proposed framework is based on offline historical student interaction data. Therefore, the learned policies were evaluated using observed transitions and

an empirical transition model rather than through direct deployment to real students.

Behavioral agreement was used to measure how frequently the learned policy selected the same action as the historical dataset:

$$\text{Behavioral Agreement} = \frac{\text{Number of matching actions}}{\text{Total evaluated actions}}. \tag{32}$$

However, behavioral agreement should not be interpreted as the true accuracy of a recommendation. A learned policy may recommend an action that differs from the historical student action while potentially producing a better learning outcome.

Similarly, the mastery completion measure used in this experiment should be interpreted as an offline proxy rather than direct evidence of educational improvement in a real learning environment. These limitations are important when interpreting the experimental results.

3.11 Summary of Experimental Findings

The main findings of the experimental analysis are summarized below:

- Under the single training seed used for the headline comparison (random state 42), Q-Learning achieved a higher test average return (30.04) than Monte Carlo Control (28.52), higher logged-action agreement (87.88% vs. 80.85%), higher empirical-best agreement (92.67% vs. 85.49%), and a higher mastery completion proxy (46.8% vs. 44.2%, compared with 42.4% for the Random baseline).

- Monte Carlo Control required less training time under this seed (0.37s vs. 0.50s for Q-Learning).

- Averaged over five random seeds, Q-Learning retains a modest but consistent advantage in test average return (Q-Learning 31.88 ± 0.92, Monte Carlo Control 30.86 ± 0.78), while the two algorithms are not reliably distinguishable on mastery success rate ($43.1\% \pm 2.7\%$ vs. $42.0\% \pm 2.4\%$). The multi-seed advantage for Q-Learning is smaller than the single-seed headline result suggested, but does not disappear.

- The hyperparameter analysis (Section 3.6) shows that neither algorithm's "main" configuration is its best-performing configuration in the tested grid, though the differences between configurations are mostly within one standard deviation of each other and the two algorithms' configuration grids overlap substantially.

- The ablation study (Section 3.7) shows that removing the weak-skill learning-history dimension produces by far the largest and most seed-robust improvement in Q-Learning's test average return (+6.31), while removing skill mastery – which appeared impactful under a single seed – has no

reliable effect once averaged over seeds. This points to state-space fragmentation from the high-cardinality weak-skill dimension as the primary limitation of the current state representation.

- Both reinforcement learning algorithms performed better than uninformed random action selection, though the improvement – particularly for Monte Carlo Control – is modest (4.4 and 1.8 percentage points over random for Q-Learning and Monte Carlo Control, respectively).

Overall, the results support reinforcement learning as a viable framework for modeling sequential educational interactions, and provide evidence, robust across five random seeds, of a modest advantage for Q-Learning over Monte Carlo Control in average return, though not in mastery success rate. outcomes.

3.12 Code Availability

The complete implementation of the proposed reinforcement learning framework, including data preprocessing, question-bundle construction, leakage-safe state generation, RL transition construction, Q-Learning, Monte Carlo Control, baseline comparison, and evaluation, is available at:

`https://github.com/Shamazda/Adaptive-Personalized-Learning-Recommendation-Using-Reinforcement-Learning`

4 Conclusion

This study investigated personalized learning recommendation as an offline reinforcement learning problem using student interaction data from the EdNet-KT3 dataset. A bundle-level learning environment was constructed in which historical student interactions were transformed into sequential states, observed learning actions, rewards, and subsequent states. Particular attention was given to temporal information leakage by constructing each decision state using only information available before the corresponding learning activity. This provided a more appropriate representation of the information that would be available to a recommendation system at decision time.

Two classical model-free reinforcement learning approaches, Q-Learning and Monte Carlo Control, were implemented and evaluated under the same empirical offline environment. A Random policy was also considered as a baseline. The experimental analysis included student-level data splitting, hyperparameter sensitivity analysis, convergence analysis, ablation experiments, and evaluation across multiple random seeds. These experiments provided a systematic comparison of the two algorithms under a common experimental framework.

The multi-seed results showed that Q-Learning achieved a mean test return of 31.88 ± 0.92, compared with 30.86 ± 0.78 for Monte Carlo Control. The corresponding Mastery Completion Proxy values were $43.1\% \pm 2.7\%$ for Q-Learning and $42.0\% \pm 2.4\%$ for Monte Carlo Control. These results indicate that both

approaches were capable of learning non-trivial policies from the historical interaction trajectories, while Q-Learning demonstrated a modest advantage in cumulative return under the evaluated configuration.

The ablation analysis further demonstrated that the representation of the learner's state has an important influence on tabular RL performance. In particular, removing the learning-history component produced a higher test return than the full state representation under the evaluated configuration. This finding suggests that additional historical features do not necessarily improve tabular RL performance when the available data are limited, as a richer representation can result in a more fragmented state space and fewer observations per state-action combination.

Overall, the findings demonstrate the feasibility of transforming EdNet-KT3 historical interaction data into an offline RL environment for investigating adaptive learning policies. However, the results should be interpreted as evidence of policy performance within the constructed offline evaluation environment, rather than direct evidence that the recommendations improve actual student learning. The study therefore provides a foundation for future research involving richer learner representations, larger datasets, more sophisticated offline RL methods, and eventually controlled online evaluation with real learners.

Despite the promising results, several limitations should be considered when interpreting the findings:

1. **Offline Nature of the Evaluation:** The most important limitation is that the study uses historical student interaction data rather than an interactive learning environment. The RL agent cannot actually recommend an activity to a student and observe the resulting learning outcome. Consequently, the experiments evaluate policies using an empirical environment derived from historical trajectories rather than measuring the effect of recommendations through real student interaction.

2. **Lack of Counterfactual Outcomes:** EdNet-KT3 records the activities that students actually performed, but it does not provide the outcomes that would have occurred if students had selected different activities. For example, if a student historically performed a practice activity, the dataset cannot directly tell us what the student's performance would have been if a revision activity had instead been recommended. Therefore, the study cannot establish causal superiority of one recommendation over another.

3. **Limited Student Sample:** For computational feasibility, the experimental pipeline used a relatively small subset of the available EdNet data, consisting of 100 selected student interaction files. Although student-level splitting was used to separate training, validation, and testing students, the limited sample size restricts the generalizability of the findings to the broader population represented by EdNet.

4. **Restricted Action Space:** The final RL environment primarily uses observable question-based activity categories such as quiz, practice, and

revision. Although EdNet-KT3 contains lecture and explanation interactions, these activities were not assigned artificial question outcomes. This decision avoids creating unreliable counterfactual labels, but it also means that the action space does not represent the complete range of educational activities available in the original platform.

5. **Simplified State Representation:** The proposed state representation uses discretized learning characteristics such as recent performance, skill mastery, and learning history. Although this makes the problem suitable for tabular Q-Learning and Monte Carlo Control, it provides only a simplified representation of a student's actual learning condition. Important factors such as long-term knowledge evolution, time between sessions, difficulty of learning materials, motivation, engagement, and individual learning preferences are not fully represented.

6. **Reward as a Proxy for Learning Outcome:** The reward function is derived from observed learning performance and therefore acts as a proxy for learning progress rather than a direct measurement of educational achievement. A higher cumulative reward in the constructed environment does not necessarily imply greater long-term knowledge retention or improved academic performance.

7. **Limited Algorithmic Scope:** The study focuses on two classical RL approaches: Q-Learning and Monte Carlo Control. While these algorithms are appropriate for investigating the proposed tabular environment, more advanced methods may be better suited to large or continuous state spaces and offline decision-making problems.

8. **Limited Hyperparameter Search:** Although multiple hyperparameter configurations were evaluated, the search space was relatively limited. Therefore, the best-performing configuration identified in the experiments should not be interpreted as a globally optimal configuration for the algorithms.

9. **Evaluation Metrics:** Metrics such as behavioral agreement and mastery completion are useful for evaluating the constructed environment but do not fully capture the quality of personalized educational recommendations. In particular, behavioral agreement measures similarity to historical student behavior rather than whether the recommended action is objectively better for the student.

Several directions can be explored to extend this research in the future:

1. **Use a Larger Student Population:** Future experiments should utilize a substantially larger portion of the EdNet dataset. Increasing the number of students and trajectories would provide more state-action observations and could reduce the sparsity problem encountered by tabular RL methods.

2. **Develop Richer Student-State Representations:** Future work could replace the current discretized state representation with richer sequential representations. Recurrent models such as LSTMs or GRUs, as well as transformer-based student modeling approaches, could capture longer-term dependencies in student learning histories.

   For example, instead of representing a student using only a few discrete values:

   $$s_t = (\text{accuracy}, \text{mastery}, \text{history}) \tag{33}$$

   Future models could represent the student's complete recent interaction sequence:

   $$s_t = (x_{t-k}, x_{t-k+1}, \ldots, x_{t-1}) \tag{34}$$

   where each x_i contains information about the corresponding learning interaction.

3. **Expand the Recommendation Action Space:** Future work could investigate a broader set of learning activities, including lectures, explanations, practice questions, revision materials, and different levels of question difficulty. However, this should be done only when reliable outcome signals can be associated with the corresponding activities.

4. **Investigate Advanced Offline RL Algorithms:** The current study focuses on classical tabular RL algorithms. Future research could investigate algorithms specifically designed for offline decision-making, such as Conservative Q-Learning (CQL), Batch-Constrained Q-Learning (BCQ), or other offline policy-learning approaches. These methods may be more suitable for learning from fixed historical datasets where arbitrary exploration is not possible.

5. **Improve Reward Modeling:** A more sophisticated reward function could combine multiple aspects of learning behavior rather than relying primarily on immediate performance. Future reward models could incorporate:

   - improvement in knowledge mastery,
   - sustained performance,
   - question difficulty,
   - completion behavior,
   - learning efficiency,
   - long-term retention.

   This could provide a closer approximation to meaningful educational outcomes.

- **Investigate Counterfactual Evaluation:** Because historical datasets do not directly provide counterfactual outcomes, future work could investigate causal or counterfactual evaluation techniques. Such approaches could help estimate the potential outcome of recommending activities that were not historically selected by a student.
- **Conduct Online or Human-Subject Evaluation:** The most important future direction is to move from offline evaluation toward controlled online evaluation. A deployed recommendation system could provide learning activities to real students and measure subsequent learning outcomes. Such an evaluation would provide stronger evidence about whether RL-based recommendations actually improve student learning.
- **Explore Personalized Policies:** Future research could investigate whether policies can be specialized according to different learner characteristics. For example, students with consistently low mastery may benefit from different activity sequences than students who demonstrate high mastery. This could lead to more individualized policies rather than a single policy applied uniformly across all students.
- **Evaluate Long-Term Learning Outcomes:** Future experiments should consider longer-term educational outcomes rather than evaluating only immediate or short-term rewards. Knowledge retention, repeated assessment performance, and progress across multiple topics could provide stronger evidence of the educational effectiveness of the learned policies.

References

[1] Xueying Tang, Yunxiao Chen, Xiaoou Li, Jingchen Liu, and Zhiliang Ying. A reinforcement learning approach to personalized learning recommendation systems. *British Journal of Mathematical and Statistical Psychology*, 72(1):108–135, 2019.

[2] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, USA, 2nd edition, 2018.

[3] Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas J. Guibas, and Jascha Sohl-Dickstein. Deep knowledge tracing. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

[4] Chang Tan, Ruimin Han, Ronghai Ye, and Kaiquan Chen. Adaptive learning recommendation strategy based on deep Q-learning. *Applied Psychological Measurement*, 44(4):251–266, 2020.

[5] Yunxiao Chen, Xiaoou Li, Jingchen Liu, and Zhiliang Ying. Recommendation system for adaptive learning. *Applied Psychological Measurement*, 42(1):24–41, 2018.

[6] Andreas Riedmann, Philip Schaper, and Birgit Lugrin. Reinforcement learning in education: A systematic literature review. *International Journal of Artificial Intelligence in Education*, 2025.

[7] Youngduck Choi, Youngnam Lee, Dongmin Shin, Junghyun Cho, Seoyon Park, Seewoo Lee, Jineon Baek, Chan Bae, Byungsoo Kim, and Jaewe Heo. EdNet: A large-scale hierarchical dataset in education. In *Proceedings of the 21st International Conference on Artificial Intelligence in Education (AIED)*, 2020.